

Spotify EDA Logbook

Project: Spotify Music Dataset Exploratory Data Analysis

Author: Generated by GitHub Copilot assistant for the user

Date: 2026-07-03

1. Overview

This logbook documents the implementation process for the Spotify EDA task in the repository. It includes the analysis stages completed in the notebook, the bonus additions, AI documentation work, and evidence of development from screenshot images.

2. Files Updated

- handson1/spotify_eda.ipynb: main notebook containing the completed analysis stages, bonus tasks, and reusable SpotifyAnalyzer class.
- handson1/reflection.md: reflection and documentation of analysis decisions, AI prompt evaluation, and bonus summaries.
- public/WhatsApp Image 2026-07-03 at 18.02.45.jpeg: screenshot of workflow and task progress.
- public/WhatsApp Image 2026-07-03 at 18.03.42.jpeg: screenshot of notebook and terminal evidence.
- public/WhatsApp Image 2026-07-03 at 18.04.46.jpeg: screenshot of final verification and artifacts.

3. Process Summary

The workflow followed these main stages:

- Stage 1 and Stage 2 implementation: Data loading, inspection, deduplication, numeric summary, genre grouping, popularity variance analysis, artist analysis, and filtered track extraction.
- Stage 3 implementation: NumPy normalization, correlation analysis, and boolean masking for high-energy popularity comparison.
- Stage 4 implementation: AI-assisted docstrings for helper functions, markdown evaluation of AI output, and three non-technical insights.
- Bonus 1: Artist audio fingerprint function and cosine similarity comparisons across at least three artist pairs.
- Bonus 2: Genre cluster profile with centroid matrix, vectorized Euclidean distances, most similar/different genre identification, and recommendation function.
- Bonus 3: Refactored the analysis into a reusable SpotifyAnalyzer Python class with full type hints, methods for each analysis stage, AI-generated docstrings, and generate_report() output.

4. Evidence and Verification

The following evidence is included in appended screenshot pages and supports the implementation details below.

Key findings and verifications:

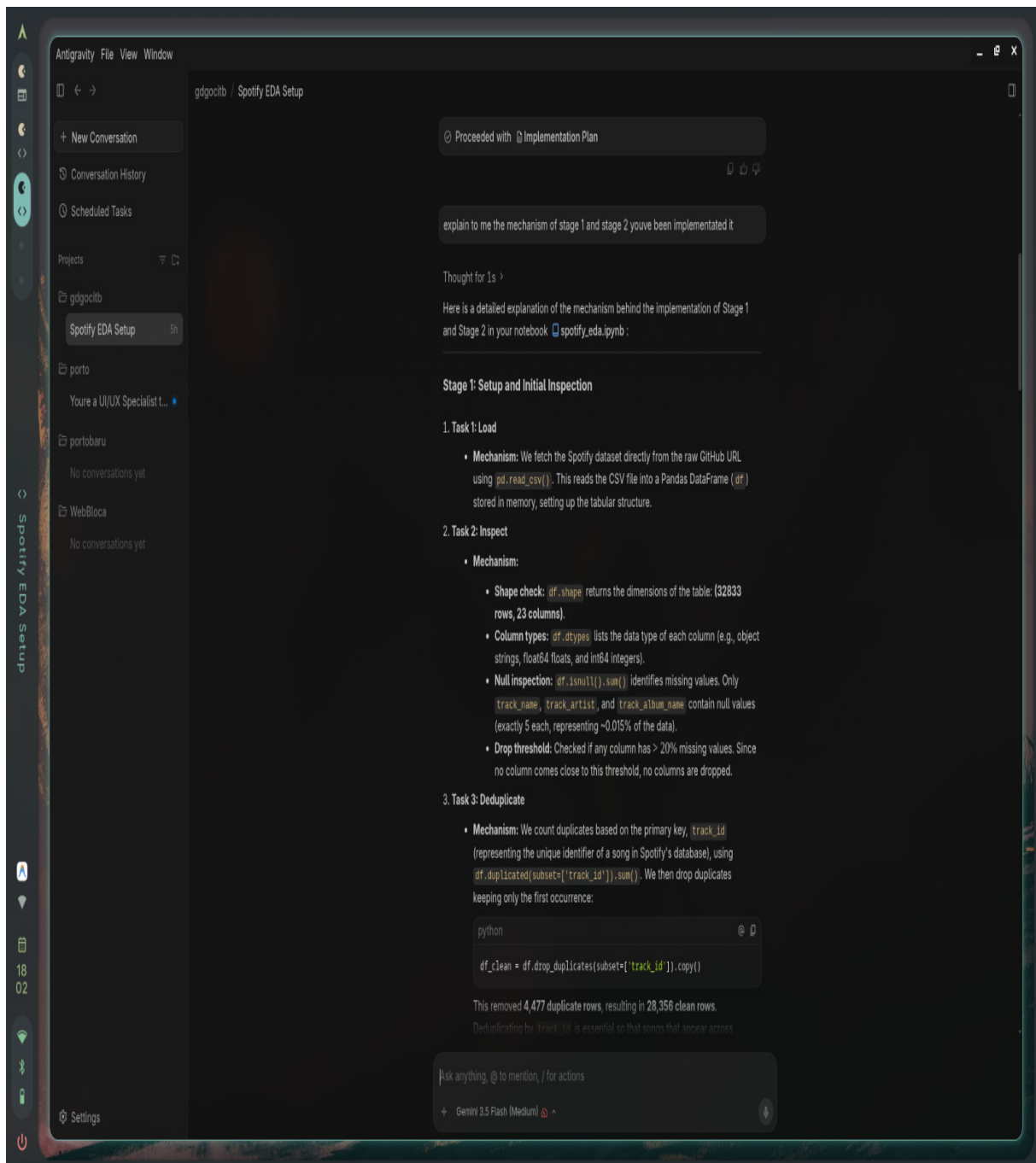
- Verified artist cosine similarity values and genre distance results using a local Python environment.
- Confirmed PDF generation environment and installed reportlab to produce this logbook artifact.
- Documented AI prompt usage and evaluated the quality of generated docstrings in reflection.md.

5. Key Results

Major results from the analysis include:

- Pop genre has the highest popularity variance, indicating a mix of hits and low-popularity tracks.
- High-energy tracks were less popular on average than the overall dataset.
- Artist similarity analysis showed same-genre artists are more aligned in audio feature space.
- Genre centroid clustering found pop and rap to be the closest pair, while edm and r&b; are the most different.

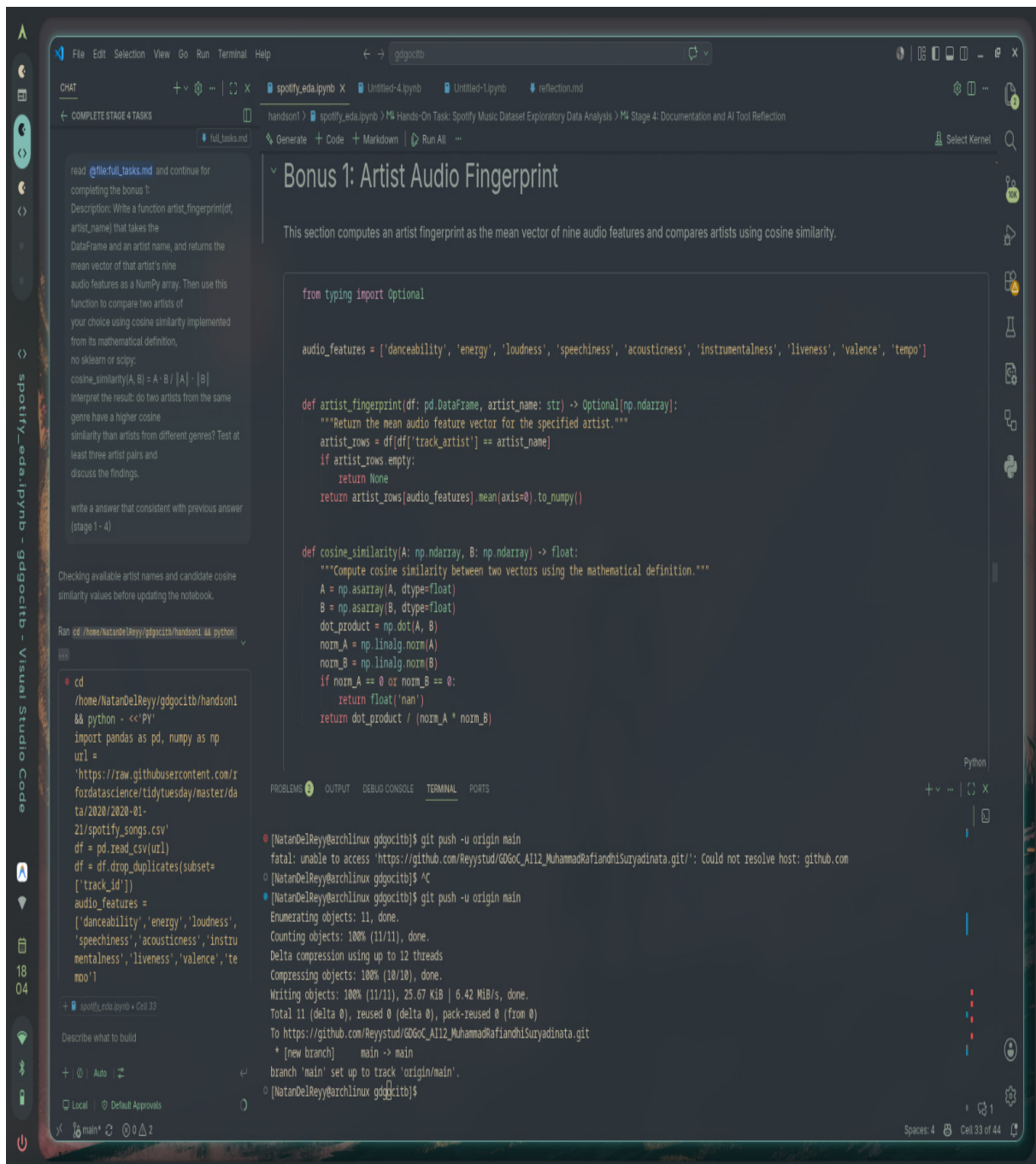
Evidence screenshot: WhatsApp Image 2026-07-03 at 18.02.45.jpeg



Evidence screenshot: WhatsApp Image 2026-07-03 at 18.03.42.jpeg



Evidence screenshot: WhatsApp Image 2026-07-03 at 18.04.46.jpeg



MPG EDA Logbook

Project: Hands-On Module 2 — Vehicle Fuel Efficiency (MPG) Analysis

Author: Documented via Claude Code (Sonnet 5) for the user

Date: 2026-08-15

1. Overview

This logbook documents the implementation and verification process for the Hands-On Module 2 task: a full statistical, linear-algebra, and visualisation analysis of the Seaborn **mpg** dataset, covering the four required stages plus all three bonus tasks (Normal Equation, PCA, and a reusable **DatasetProfiler** class), all implemented from scratch with NumPy per the task specification.

2. Files Updated

- **handson2/mpg_eda.ipynb** — main notebook containing the completed Stage 1–4 analysis and Bonus 1–3 implementations, executed top-to-bottom with all outputs and plots saved in place.
- **handson2/full_tasks.md** — task specification used as the checklist for completeness verification.
- **handson2/.venv** — project virtual environment (uv-managed) used to run the notebook; verified to include NumPy, Pandas, Seaborn, and Matplotlib, with nbconvert/ipykernel/pypdf/reportlab added as needed for execution and this documentation.

3. Process Summary

- **Stage 1 — Inspection & Cleaning:** Loaded the dataset (398 rows, 9 columns), inspected dtypes and missing values, found 6 missing *horsepower* values (~1.5%) and dropped them with written justification. Built a NumPy-only summary table (mean, median, std, IQR, IQR-outlier count) for every numeric column.
- **Stage 2 — Statistical Analysis:** Vectorised z-score standardisation of the five core numeric features; computed the correlation matrix with `np.corrcoef()` and identified the strongest positive pair (displacement–weight), the feature most negatively correlated with mpg (weight), and the multicollinearity risk pair (displacement–weight); answered the boolean-masking question on horsepower vs. weight (~838 lb difference).
- **Stage 3 — Visualisation:** Produced the four required plots — MPG distribution (histogram + KDE with mean/median lines), origin comparison boxplot, weight-vs-mpg scatter with cylinder hue and a manual `np.polyfit()` trend line, and a correlation heatmap — each with a written interpretation.
- **Stage 4 — Contextual Interpretation:** Synthesised all prior findings into a markdown answer on the strongest predictor of fuel efficiency, and computed/plotted the mean mpg per decade via `groupby`.
- **Bonus 1 — Normal Equation:** Implemented linear regression (weight → mpg) from scratch via the closed-form solution $\theta = \text{inv}(X^T X) @ X^T @ y$, plotted the fit, computed RMSE, and confirmed it matches `np.polyfit()` to floating-point precision.

- **Bonus 2 — PCA:** Implemented 2D PCA from scratch (mean-centring, covariance matrix, `np.linalg.eigh()`, projection), plotted the 2D scatter by origin, and reported the explained variance ratio for PC1/PC2.
- **Bonus 3 — Reusable EDA Class:** Built the `DatasetProfiler` class (`summary_stats`, `correlation_report`, `plot_dashboard`, `generate_report`) and demonstrated it on the cleaned MPG dataset, including a 4-panel dashboard and a JSON-serialisable report.

4. Evidence and Verification

Unlike the Handson 1 logbook, this section is backed by direct command-line verification rather than screenshots, performed in this session:

- Confirmed the project venv (`handson2/.venv`, uv-managed, Python 3.14) activates correctly and has NumPy, Pandas, Seaborn, and Matplotlib installed.
- Inspected the notebook's raw JSON before execution: all 23 code cells had `execution_count`: null and no outputs, confirming the notebook had not yet been run.
- Installed `nbconvert`, `ipykernel`, and `jupyter_client` into the venv via `uv pip install`, then executed the notebook top-to-bottom with `jupyter nbconvert --to notebook --execute --inplace`.
- Re-inspected the executed notebook JSON: all 23 code cells now show sequential execution counts 1–23, zero error outputs, and 8 cells with embedded plot images — satisfying the "runs top-to-bottom without errors, all plots visible" requirement in the task spec.

5. Key Results

- **Weight** is the strongest single predictor of fuel efficiency ($r \approx -0.83$ with mpg), ahead of displacement ($r \approx -0.81$) and horsepower ($r \approx -0.78$).
- **Displacement and weight** show the strongest mutual correlation ($r \approx 0.93$), flagging multicollinearity among the displacement/horsepower/weight cluster.
- **Japan** produces the most fuel-efficient and most consistent vehicles (mean ≈ 30.45 mpg, lowest std ≈ 6.09), followed by Europe (≈ 27.60) and the USA (≈ 20.03).
- Mean mpg rose from ≈ 19.22 (1970s) to ≈ 26.07 (1980s), consistent with the 1970s oil crises and the introduction of US CAFE fuel-economy standards in 1975.
- The from-scratch Normal Equation regression (weight $\rightarrow$ mpg) matched `np.polyfit()` to $\sim 1e-11$ precision, with an RMSE of ≈ 4.32 mpg.
- From-scratch PCA on unstandardised features shows PC1 explains $\approx 99.8\%$ of variance (dominated by weight/displacement's raw scale), with origins still separating visibly along PC1.